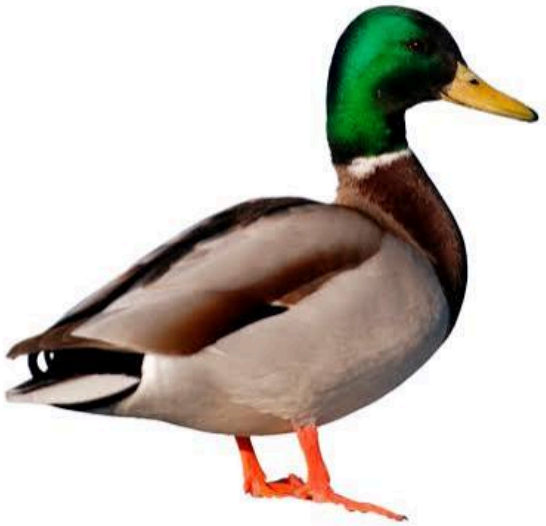
Ciência de Dados Aplicada ao Direito I

APS

O desafio: Claude Code para advogados



Prof. Bruno da Cunha de Oliveira - 2026/2

Um problema do milênio

Insper

Os sete Problemas do Milênio

- Em 2000, o Instituto Clay listou sete problemas abertos da matemática e ofereceu US$ 1 milhão por cada um.

Os sete Problemas do Milênio

- Em 2000, o Instituto Clay listou sete problemas abertos da matemática e ofereceu US$ 1 milhão por cada um.

- Hipótese de Riemann (1859) · Poincaré (1904) · Navier-Stokes (anos 1930) · Hodge (1950) · Yang-Mills (1954) · Birch e Swinnerton-Dyer (anos 1960) · P versus NP (1971).

Os sete Problemas do Milênio

- Em 2000, o Instituto Clay listou sete problemas abertos da matemática e ofereceu US$ 1 milhão por cada um.

- Hipótese de Riemann (1859) · Poincaré (1904) · Navier-Stokes (anos 1930) · Hodge (1950) · Yang-Mills (1954) · Birch e Swinnerton-Dyer (anos 1960) · P versus NP (1971).

- Os mais famosos da matemática: o mais novo tem 55 anos, o mais velho, 167. Gerações inteiras tentaram e não conseguiram.

Os sete Problemas do Milênio

- Em 2000, o Instituto Clay listou sete problemas abertos da matemática e ofereceu US$ 1 milhão por cada um.

- Hipótese de Riemann (1859) · Poincaré (1904) · Navier-Stokes (anos 1930) · Hodge (1950) · Yang-Mills (1954) · Birch e Swinnerton-Dyer (anos 1960) · P versus NP (1971).

- Os mais famosos da matemática: o mais novo tem 55 anos, o mais velho, 167. Gerações inteiras tentaram e não conseguiram.

- Só Poincaré caiu, em 2003, por Grigori Perelman, que recusou o prêmio. Navier-Stokes, o dos fluidos, era o próximo da fila.

September 8, 2026 Research Publication

On the Navier–Stokes Millennium Prize Problem

Read the paper

Link to Lean formalized proof >

Quem resolveu, e em quanto tempo

- A fofoca já corria há semanas: um Problema do Milênio teria caído para LLMs. Hoje saiu o post.

Quem resolveu, e em quanto tempo

- A fofoca já corria há semanas: um Problema do Milênio teria caído para LLMs. Hoje saiu o post.

- Um grupo de agentes, rodando um modelo interno da OpenAI mais capaz que o GPT-6 Astra, o modelo mais forte disponível ao público.

Quem resolveu, e em quanto tempo

- A fofoca já corria há semanas: um Problema do Milênio teria caído para LLMs. Hoje saiu o post.
- Um grupo de agentes, rodando um modelo interno da OpenAI mais capaz que o GPT-6 Astra, o modelo mais forte disponível ao público.
- A resolução chegou no sábado, 05/09, 88 horas depois de os primeiros agentes serem lançados.

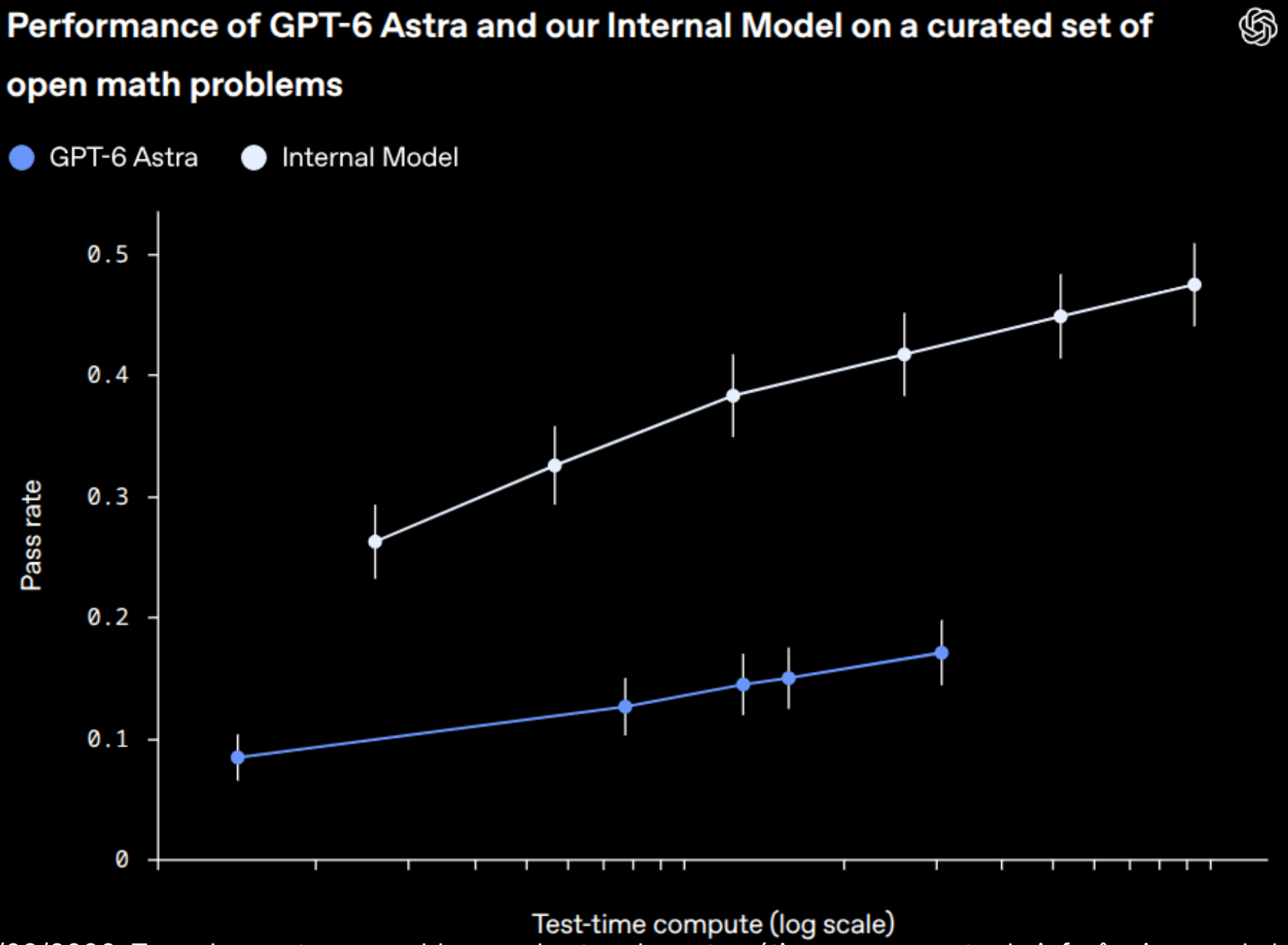
Quem resolveu, e em quanto tempo

- A fofoca já corria há semanas: um Problema do Milênio teria caído para LLMs. Hoje saiu o post.
- Um grupo de agentes, rodando um modelo interno da OpenAI mais capaz que o GPT-6 Astra, o modelo mais forte disponível ao público.
- A resolução chegou no sábado, 05/09, 88 horas depois de os primeiros agentes serem lançados.
- A prova foi formalizada e verificada em Lean em mais 17 horas, pelo GPT-6 Astra.

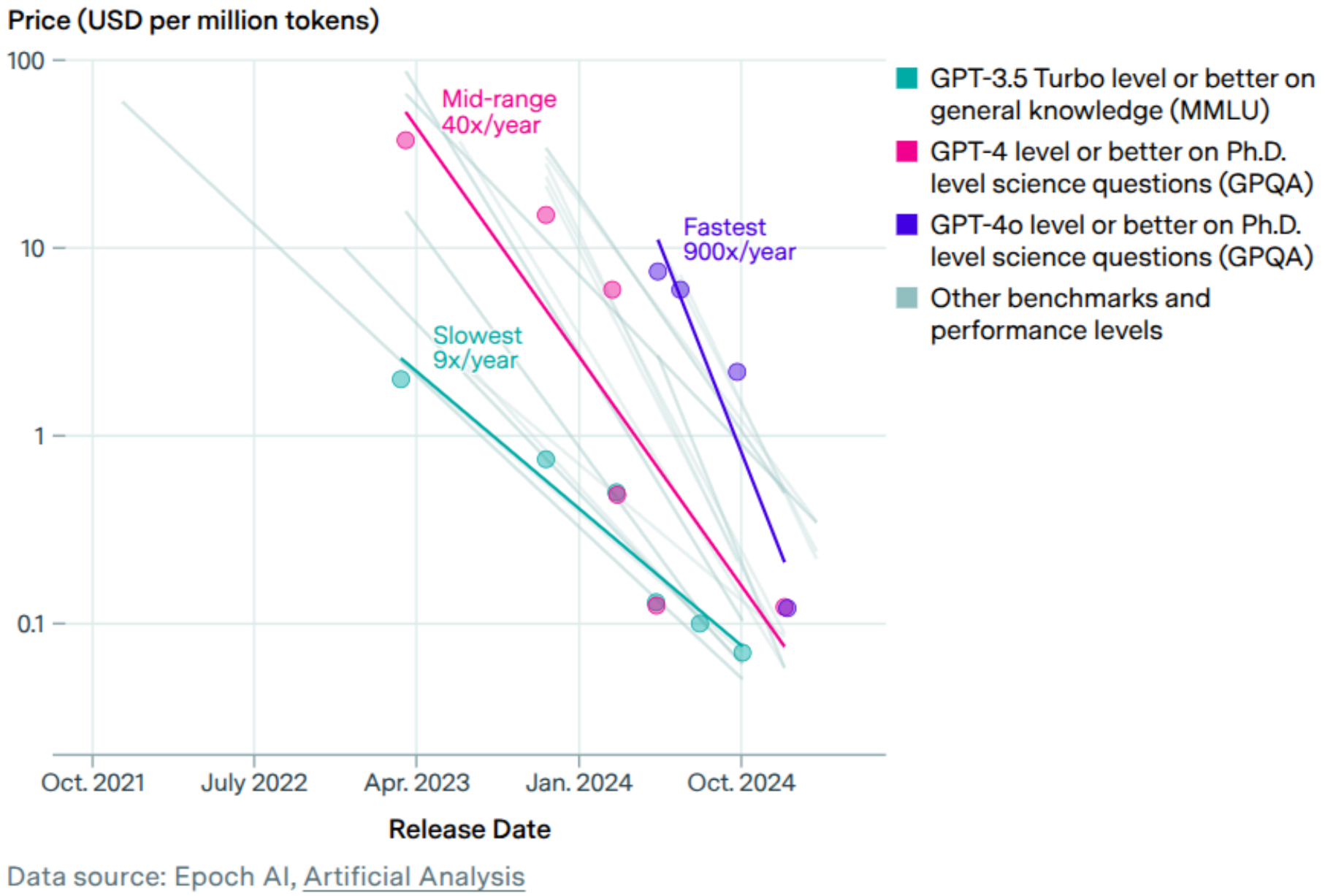
Quanto custou, ao preço do GPT-6 Astra

	Mensagens	Tokens de saída	Estimativa
Todos os problemas tentados	4,9 milhões	300 bilhões	US$ 15 milhões
Navier-Stokes	2,7 milhões	130 bilhões	US$ 6,5 milhões
Prêmio do Instituto Clay			US$ 1 milhão

Só tokens de saída, a US$ 50 por milhão, que é o preço do Astra na API. O modelo usado é mais caro e a entrada não foi divulgada: a conta é o piso.



Fonte: OpenAI, 08/09/2026. Taxa de acerto em problemas abertos de matemática por compute de inferência, escala logarítmica.



Fonte: Epoch AI e Artificial Analysis. Preço por milhão de tokens para manter um nível fixo de desempenho, escala logarítmica.

Eles estão só começando

- Não importa o quão inteligente você acha que os modelos já são: a curva ainda sobe, e o preço ainda cai.

Eles estão só começando

- Não importa o quão inteligente você acha que os modelos já são: a curva ainda sobe, e o preço ainda cai.

- Esse volume de recurso está indo para a matemática. As pessoas mais inteligentes do mundo ainda não estão olhando para o Direito.

Eles estão só começando

- Não importa o quão inteligente você acha que os modelos já são: a curva ainda sobe, e o preço ainda cai.
- Esse volume de recurso está indo para a matemática. As pessoas mais inteligentes do mundo ainda não estão olhando para o Direito.
- Tem gente boa pensando IA em Direito. Mesmo assim, há muita ideia inexplorada, e é aí que entra a APS.

O que é um harness

- O modelo, sozinho, recebe texto e devolve texto. Ele não abre arquivo, não roda comando, não lembra da conversa de ontem.

O que é um harness

- O modelo, sozinho, recebe texto e devolve texto. Ele não abre arquivo, não roda comando, não lembra da conversa de ontem.

- O harness é tudo que fica em volta: prompt de sistema, ferramentas, permissões, memória e o ciclo de agir, observar o resultado e agir de novo.

O que é um harness

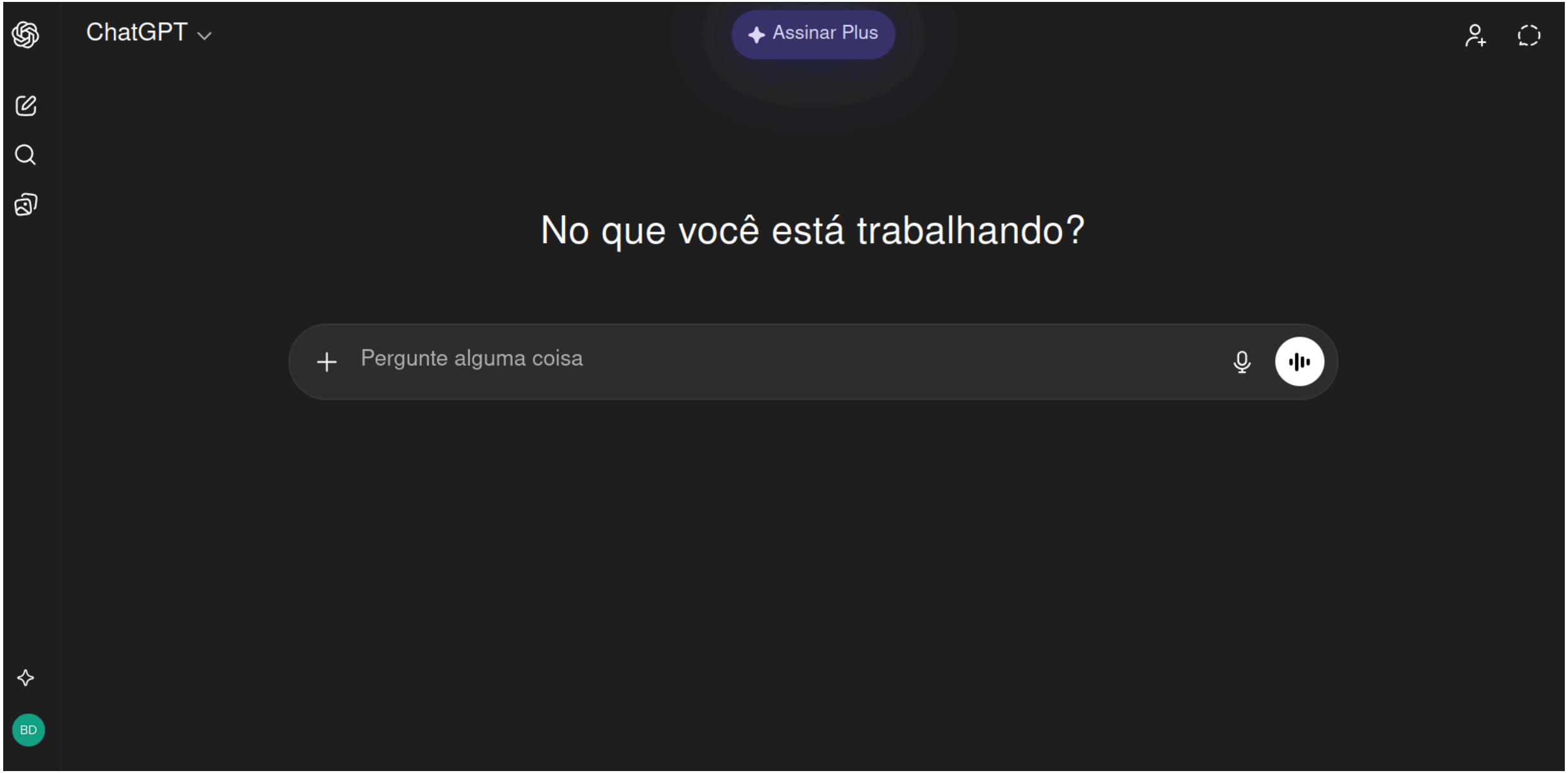
- O modelo, sozinho, recebe texto e devolve texto. Ele não abre arquivo, não roda comando, não lembra da conversa de ontem.

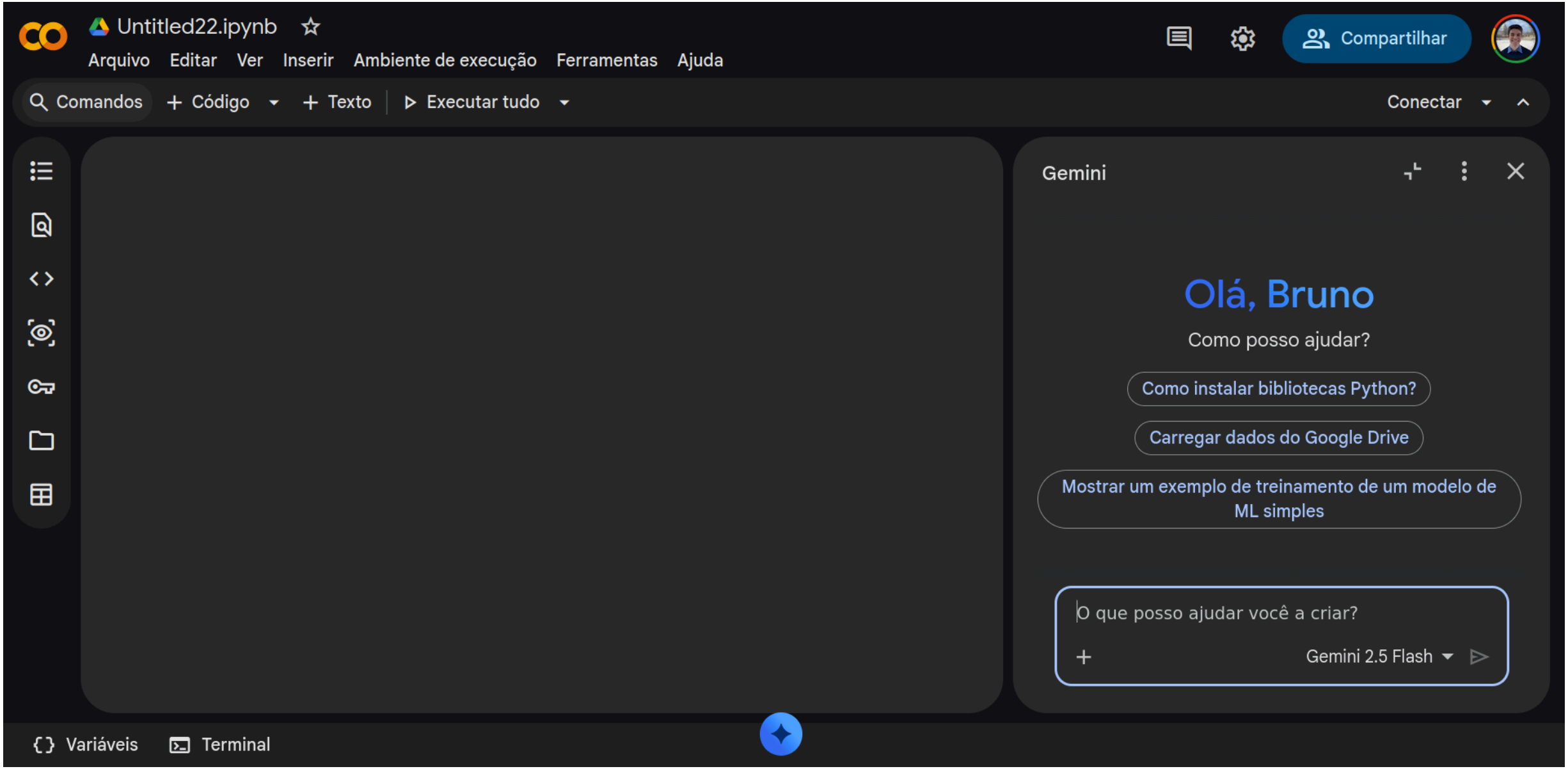
- O harness é tudo que fica em volta: prompt de sistema, ferramentas, permissões, memória e o ciclo de agir, observar o resultado e agir de novo.

- O Claude Code é um harness sobre um modelo. Os agentes do Navier-Stokes também. A APS pede que vocês desenhem o harness, não o modelo.

Construir o Claude Code para advogados

Insper


ChatGPT
Assinar Plus
No que você está trabalhando?
Pergunte alguma coisa
BD


Untitled22.ipynb
Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda
Compartilhar
Comandos + Código + Texto Executar tudo
Conectar
Gemini
Olá, Bruno
Como posso ajudar?
Como instalar bibliotecas Python?
Carregar dados do Google Drive
Mostrar um exemplo de treinamento de um modelo de ML simples
O que posso ajudar você a criar?
Gemini 2.5 Flash
Variáveis
Terminal

• Computing...

Built for > engineers

Work with Claude directly in your codebase. Build, debug, and ship from your terminal, IDE, Slack, or the web. Describe what you need, and Claude handles the rest.

Get Claude Code

```
curl -fsSL https://claude.ai/install.sh | bash
```

Or read the documentation

Sponsored by: WorkOS — The infrastructure fast-growing B2B companies use to sell to Enterpri

It genuinely feels to me like GPT-5.2 and Opus 4.5 in November represent an inflection point - one of those moments where the models get incrementally better in a way that tips across an invisible capability line where suddenly a whole bunch of much harder coding problems open up.

Posted 4th January 2026 at 11:21 pm

MARKETS

Claude the conqueror: The AI chatbot keeps wiping out billions from exposed tech stocks

By Jennifer Sor + Follow



AD

AI • ANTHROPIC

Anthropic's Claude triggered a trillion-dollar selloff. A new upgrade could make things worse

By **Beatrice Nolan**
Tech Reporter

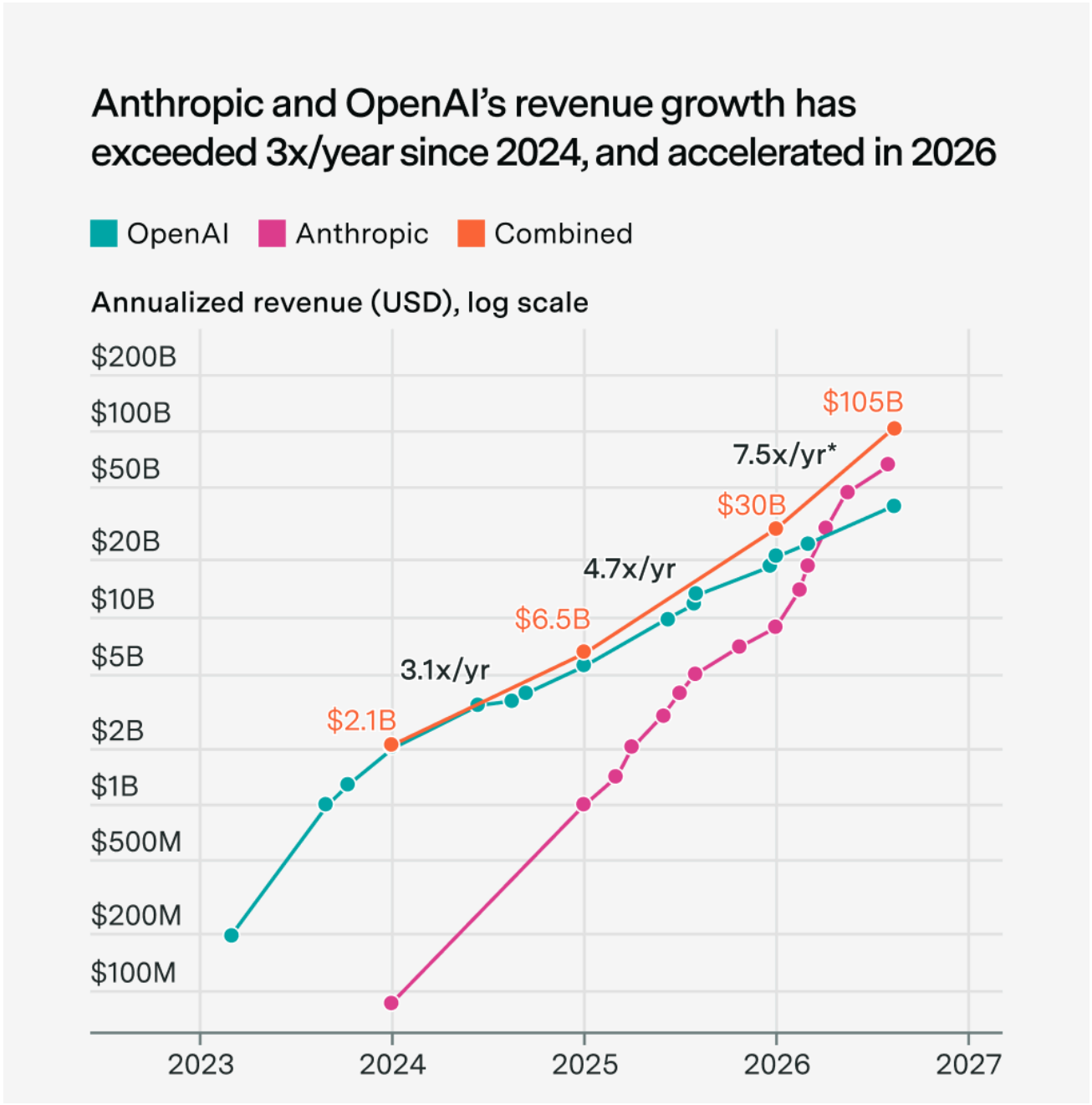
February 6, 2026, 10:58 AM ET

Add us on



Dario Amodei, cofounder and CEO of Anthropic. The company just debuted its new Claude Opus 4.6 model, days after an update to its Claude Cowork tool sparked a massive selloff in the shares of traditional software companies.
CHRIS RATCLIFFE—BLOOMBERG/GETTY IMAGES


OpenAI
Entrar
Experimente o ChatGPT ↗
Codex
Experimente com o Free e o Go, ou aproveite limites de taxa dobrados em outros planos por tempo limitado.
Experimente na sua IDE
Entre na lista de espera do aplicativo do Codex



Fonte: Epoch AI, CC-BY, epoch.ai. Receita anualizada em dólares, escala logarítmica.

Versão 2.1.263

- O Claude Code está na versão 2.1.263, e amanhã não vai estar mais: muda todo dia.

Versão 2.1.263

- O Claude Code está na versão 2.1.263, e amanhã não vai estar mais: muda todo dia.

- Se em código, onde ele nasceu, ainda há muito o que melhorar, aprimorar e afinar, imaginem no Direito.

Versão 2.1.263

- O Claude Code está na versão 2.1.263, e amanhã não vai estar mais: muda todo dia.
- Se em código, onde ele nasceu, ainda há muito o que melhorar, aprimorar e afinar, imaginem no Direito.
- No Direito ainda não existe um killer app. O espaço para a proposta de vocês é esse.

Claude Code na prática

Insper

O que é o Claude Code

- Um assistente de IA que trabalha nos arquivos de um projeto: você descreve o que quer e ele escreve e altera código.

O que é o Claude Code

- Um assistente de IA que trabalha nos arquivos de um projeto: você descreve o que quer e ele escreve e altera código.

- Ele executa comandos e testes, observa os resultados e pode corrigir o que deu errado.

O que é o Claude Code

- Um assistente de IA que trabalha nos arquivos de um projeto: você descreve o que quer e ele escreve e altera código.
- Ele executa comandos e testes, observa os resultados e pode corrigir o que deu errado.
- Você acompanha o trabalho e confere se o resultado atende ao pedido.

A proposta do grupo

- Proponham um assistente de programação por IA cujo público-alvo seja todos os advogados, de qualquer área de atuação.

A proposta do grupo

- Proponham um assistente de programação por IA cujo público-alvo seja todos os advogados, de qualquer área de atuação.

- Mostrem que soluções um advogado poderá construir para o próprio trabalho com essa ferramenta.

A proposta do grupo

- Proponham um assistente de programação por IA cujo público-alvo seja todos os advogados, de qualquer área de atuação.
- Mostrem que soluções um advogado poderá construir para o próprio trabalho com essa ferramenta.
- Expliquem quais configurações do produto de vocês fariam esse público preferi-lo a um assistente genérico.

O que vocês podem configurar

Prompt de sistema

Instruções gerais: perguntar o objetivo jurídico, preservar o sigilo e pedir critérios para conferir o resultado.

O que vocês podem configurar

Prompt de sistema

Instruções gerais: perguntar o objetivo jurídico, preservar o sigilo e pedir critérios para conferir o resultado.

Skills

Instruções reutilizáveis para criar um organizador de documentos ou uma ferramenta de cronologia.

O que vocês podem configurar

Prompt de sistema

Instruções gerais: perguntar o objetivo jurídico, preservar o sigilo e pedir critérios para conferir o resultado.

Skills

Instruções reutilizáveis para criar um organizador de documentos ou uma ferramenta de cronologia.

Modo de planejamento

Propor etapas e testes antes de programar. Pedir aprovação do advogado para executar o plano.

O que vocês podem configurar

Prompt de sistema

Instruções gerais: perguntar o objetivo jurídico, preservar o sigilo e pedir critérios para conferir o resultado.

Skills

Instruções reutilizáveis para criar um organizador de documentos ou uma ferramenta de cronologia.

Modo de planejamento

Propor etapas e testes antes de programar. Pedir aprovação do advogado para executar o plano.

Permissões

Definir quando pedir autorização para alterar arquivos, executar comandos ou enviar dados.

Isso são só exemplos

- Os quatro cartões são exemplos, e a lista não fecha: o céu é o limite.

Isso são só exemplos

- Os quatro cartões são exemplos, e a lista não fecha: o céu é o limite.
- Pensem também em interface e experiência do usuário: como o advogado conversa com a ferramenta, o que ele vê e o que ele precisa aprender para usá-la.

Isso são só exemplos

- Os quatro cartões são exemplos, e a lista não fecha: o céu é o limite.
- Pensem também em interface e experiência do usuário: como o advogado conversa com a ferramenta, o que ele vê e o que ele precisa aprender para usá-la.
- O que vale é a pergunta: o que faria um advogado preferir o produto de vocês a um assistente genérico?

Vender a ideia e construir o produto

- Fase 1 · 14/10. Apresentar a proposta num site e num pitch para uma banca de advogados. Vale 15% da nota do semestre.

Vender a ideia e construir o produto

- Fase 1 · 14/10. Apresentar a proposta num site e num pitch para uma banca de advogados. Vale 15% da nota do semestre.

- Fase 2 · 18/11. Construir o produto que o grupo vendeu. Vale outros 15%.

Empresa, produto e apresentação

- Criem uma empresa fictícia, deem nome ao produto e escolham uma logo da empresa ou do produto.

Empresa, produto e apresentação

- Criem uma empresa fictícia, deem nome ao produto e escolham uma logo da empresa ou do produto.

- Entreguem os dois nomes e a logo até o início da aula de 22/09, pelo formulário no Blackboard.

Empresa, produto e apresentação

- Criem uma empresa fictícia, deem nome ao produto e escolham uma logo da empresa ou do produto.
- Entreguem os dois nomes e a logo até o início da aula de 22/09, pelo formulário no Blackboard.
- Apresentem a proposta num site público e num pitch de cinco minutos.

O site explica; o pitch convence

Site

O link deve explicar o produto, seu público e sua utilidade sem ajuda do grupo. Deve abrir sem instalação nem login.

Pitch de 5 minutos

Apresentem o produto e mostrem o site para convencer a banca como potenciais clientes. A banca não faz perguntas.

Uso de IA e outras ferramentas

- O uso de IA é liberado e incentivado no site, no pitch, na logo e no texto.

Uso de IA e outras ferramentas

- O uso de IA é liberado e incentivado no site, no pitch, na logo e no texto.
- Em cada fase, entreguem um relatório que explique o que tentaram, quais resultados observaram e por que escolheram ou descartaram ferramentas.

Uso de IA e outras ferramentas

- O uso de IA é liberado e incentivado no site, no pitch, na logo e no texto.
- Em cada fase, entreguem um relatório que explique o que tentaram, quais resultados observaram e por que escolheram ou descartaram ferramentas.
- Usem exemplos do trabalho para explicar dificuldades e aprendizado, inclusive o que fariam diferente numa próxima vez.

Entregas e prazos

Data	Entrega ou apresentação
22/09 Até o início da aula	Nome da empresa, nome do produto e logo.
13/10 Até o início da aula	URL pública do site, slides do pitch em PDF, relatório, apresentador titular e suplente.
14/10 Das 12h às 14h	Apresentação da fase 1 para a banca de advogados.

A banca: seis advogados

Direito Bancário
FAS Advogados

A banca: seis advogados

Direito Bancário
FAS Advogados

M&A
Mattos Filho

A banca: seis advogados

Direito Bancário
FAS Advogados

M&A
Mattos Filho

Concorrencial
BMA

A banca: seis advogados

Direito Bancário
FAS Advogados

M&A
Mattos Filho

Concorrencial
BMA

Direito Público
SP Parcerias

A banca: seis advogados

Direito Bancário
FAS Advogados

M&A
Mattos Filho

Concorrencial
BMA

Direito Público
SP Parcerias

Assistente Judicial
TJSP

A banca: seis advogados

Direito Bancário
FAS Advogados

M&A
Mattos Filho

Concorrencial
BMA

Direito Público
SP Parcerias

Assistente Judicial
TJSP

Advogada in-house
Alice (plano de saúde)

A nota da fase 1 pelo ranking da banca

Posição	Nota da fase 1, de 1,5
1ª	1,5, e dispensa da fase 2 com os 1,5 pontos dela
2ª	1,5
3ª	1,4
4ª	1,3
5ª	1,2
6ª	1,1
7ª a 11ª	de 0,1 a 1,0, por mim e pelas monitoras: metade o site, metade o relatório

A nota dos seis primeiros é da banca, e ninguém a altera. Da 7ª em diante, a posição não conta.

A melhor da primeira fase não precisa fazer a segunda

- A primeira colocada não precisa desenvolver o produto e recebe os 1,5 pontos da fase 2.

A melhor da primeira fase não precisa fazer a segunda

- A primeira colocada não precisa desenvolver o produto e recebe os 1,5 pontos da fase 2.

- Em troca, em 21/10, o grupo apresenta à turma como fez para ir tão bem: divisão do trabalho, referências e ferramentas, com respostas às dúvidas dos colegas.

Menos trabalho e até 2 pontos extras em uma das provas

Até 1 ponto

Integrantes da vencedora podem investir em outro grupo, apoiando-o por contrato. O ganho depende da colocação dele na fase 2.

+ até 1 ponto

Por participação: bônus das atividades da disciplina no período correspondente à prova, com nota máxima 10.

O que significa investir

- Cada integrante da vencedora pode escolher um dos outros grupos para apoiar na fase 2. Investir é facultativo.

O que significa investir

- Cada integrante da vencedora pode escolher um dos outros grupos para apoiar na fase 2. Investir é facultativo.

- O apoio é trabalho combinado: por exemplo, compartilhar métodos, ajudar a testar o produto ou discutir escolhas de ferramentas.

O que significa investir

- Cada integrante da vencedora pode escolher um dos outros grupos para apoiar na fase 2. Investir é facultativo.
- O apoio é trabalho combinado: por exemplo, compartilhar métodos, ajudar a testar o produto ou discutir escolhas de ferramentas.
- Um grupo pode receber mais de um investidor. O investidor não apresenta pelo grupo apoiado.

Como combinar o apoio

- O contrato define o apoio e a distribuição dos ganhos e perdas. Fica guardado comigo.

Como combinar o apoio

- O contrato define o apoio e a distribuição dos ganhos e perdas. Fica guardado comigo.

- O fato relevante anuncia o investimento à turma, com os termos que as partes escolherem divulgar.

Como combinar o apoio

- O contrato define o apoio e a distribuição dos ganhos e perdas. Fica guardado comigo.
- O fato relevante anuncia o investimento à turma, com os termos que as partes escolherem divulgar.
- Entreguem os dois pelo formulário no Blackboard até o início da aula de 17/11.

Resultado do investimento por colocação da investida

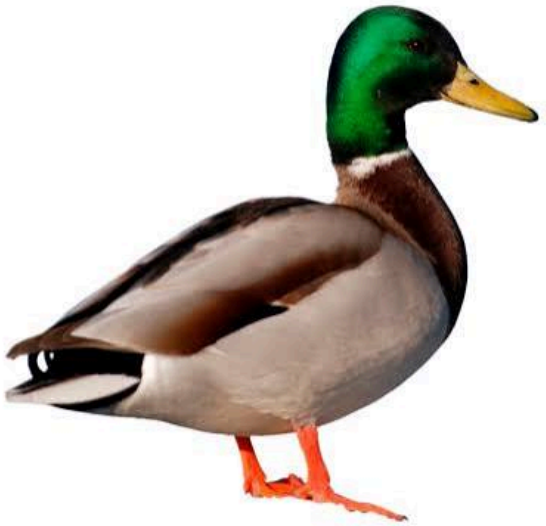
Colocação	Resultado	Colocação	Resultado
1ª	+1,0	6ª	−0,1
2ª	+0,75	7ª	−0,25
3ª	+0,5	8ª	−0,5
4ª	+0,25	9ª	−0,75
5ª	+0,1	10ª	−1,0

O resultado é repartido conforme o contrato. Valor positivo entra como bônus em uma das provas. Valor negativo é descontado da APS na média do semestre.

Ciência de Dados Aplicada ao Direito I

Aula 9

Loops, de novo: variável de iteração e `para cada` × `enquanto`



Prof. Bruno da Cunha de Oliveira - 2026/2

O que ficou da aula 8

Insper

A variável de iteração

```
decisoes = [
  {"numero_processo": "0000000-00.2026.8.26.0100", "data_sentenca": 01/09/2026, "data_transito": NULO},
  {"numero_processo": "0000000-01.2023.8.26.0100", "data_sentenca": 15/03/2023, "data_transito": 30/04/2023},
  {"numero_processo": "0000000-02.2024.8.26.0100", "data_sentenca": 10/06/2024, "data_transito": NULO}
]

PARA CADA decisao EM decisoes:
    SE decisao["data_transito"] == NULO
     E decisao["data_sentenca"] < 01/09/2024 ENTÃO
        revisar decisao
```

1. Quem escolheu o nome `decisao`?
2. O que já existia antes de o loop começar?
3. O que `decisao` vale na segunda volta? E depois que o loop acaba?

A variável de iteração

RESPOSTA

```
decisoes = [ ... ]                  ← existia antes: é a coleção

PARA CADA decisao EM decisoes:      ← decisao nasce aqui; o nome é de quem escreve
    SE decisao["data_transito"] == NULO ...
        revisar decisao
```

1. Quem escreveu o loop. `x` daria o mesmo resultado e uma leitura pior.
2. `decisoes`, a lista. O que vem depois do `EM` é sempre uma variável com uma coleção dentro.
3. O segundo dicionário. Depois do loop, o último que passou: já não é “cada decisão”, é uma só.

É a única variável que o loop renova sozinho: ninguém escreve `decisao = ...`.

A variável de controle

```
pecas = ["PETIÇÃO INICIAL", "CONTESTAÇÃO", "SENTENÇA", "APELAÇÃO"]
achou_sentenca = FALSO

ENQUANTO NÃO achou_sentenca:
    peca = a próxima peça dos autos
    olhar peca
    SE peca == "SENTENÇA" ENTÃO
        achou_sentenca = VERDADEIRO
```

Qual linha faz este loop parar? E se eu apagá-la, o que acontece quando o robô chega à sentença?

A variável de controle

RESPOSTA

```
achou_sentenca = FALSO              ← declarada ANTES do loop

ENQUANTO NÃO achou_sentenca:        ← lida na condição
    peca = a próxima peça dos autos
    SE peca == "SENTENÇA" ENTÃO
        achou_sentenca = VERDADEIRO  ← alterada DENTRO do bloco
```

A última linha. Sem ela, `achou_sentenca` nasce `FALSO` e nunca muda: `NÃO achou_sentenca` é `VERDADEIRO` para sempre, e o loop nunca para.

Declarada antes, lida na condição, alterada dentro. No `ENQUANTO`, quem escreve o loop é responsável por ele acabar.

Qual dos dois para sozinho

```
PARA CADA testemunha EM testemunhas:
    ouvir testemunha
```

```
ENQUANTO NÃO achou_sentenca:
    abrir a próxima peça dos autos
```

Um dos dois para sozinho. Qual, e por quê? E quando os dois servem, qual vocês escolhem?

Qual dos dois para sozinho

RESPOSTA

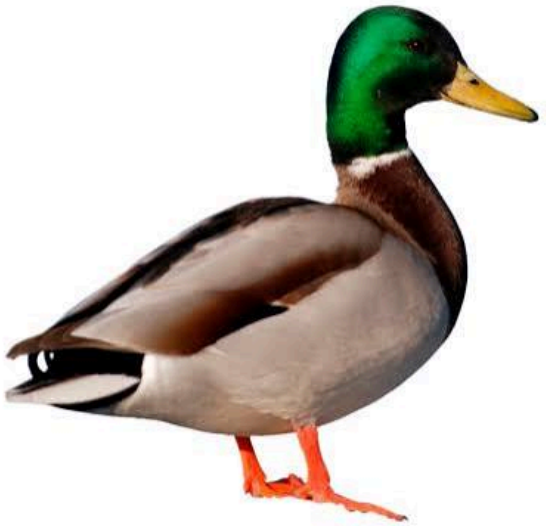
PARA CADA	ENQUANTO
Há uma coleção, e o loop passa por cada item dela.	Há uma condição, e o loop roda enquanto ela for verdadeira.
O número de voltas é o tamanho da coleção.	O número de voltas não se sabe de antemão.
Para sozinho, quando a coleção acaba.	Para quando alguma coisa dentro do bloco faz a condição virar FALSO.

Se a lista existe antes de começar, `PARA CADA`: ele acaba sozinho e não tem variável de controle para esquecer.

Ciência de Dados Aplicada ao Direito I

Aula 9

Funções, e a ideia de classe



Prof. Bruno da Cunha de Oliveira - 2026/2

Funções

Insper

A mesma conta em três lugares

Três prazos, e a Defensoria tem prazo em dobro. Escrevemos a conta três vezes:

```
SE parte_tem_prazo_em_dobro ENTÃO
    prazo_contestacao = 15 * 2
SENÃO
    prazo_contestacao = 15

SE parte_tem_prazo_em_dobro ENTÃO
    prazo_apelacao = 15 * 2
SENÃO
    prazo_apelacao = 15

SE parte_tem_prazo_em_dobro ENTÃO
    prazo_embargos = 5 * 2
SENÃO
    prazo_embargos = 5
```

Se a regra do dobro mudar, são três lugares para corrigir, e um deles vai ser esquecido.

Vamos dar um nome a isso

A conta é sempre a mesma: recebe um prazo em dias e uma resposta sobre o dobro, e devolve o prazo certo. Vamos escrever essa conta uma vez só, com um nome, de um jeito que dê para usá-la nos três lugares.

Não existe sintaxe certa ainda. O que importa é: o que entra, o que sai, e como se chama.

Vamos dar um nome a isso

RESPOSTA

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):
    SE tem_prazo_em_dobro ENTÃO
        DEVOLVER prazo_normal * 2
    SENÃO
        DEVOLVER prazo_normal

prazo_contestacao = prazo_final(15, parte_tem_prazo_em_dobro)
prazo_apelacao    = prazo_final(15, parte_tem_prazo_em_dobro)
prazo_embargos    = prazo_final(5, parte_tem_prazo_em_dobro)
```

A conta existe uma vez, com nome. Cada linha de baixo a chama com entradas diferentes e guarda o que ela devolve.

Função

Em Python: `def`

Um nome para uma sequência de instruções que recebe entradas e devolve uma saída. Escreve-se uma vez e chama-se pelo nome quantas vezes for preciso.

EXEMPLO

```
FUNÇÃO nome(entrada_1, entrada_2):
    ... o que ela faz ...
    DEVOLVER saida

resultado = nome(valor_1, valor_2)
```

Definição e chamada

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):    ← DEFINIÇÃO: escreve o que a função faz
    SE tem_prazo_em_dobro ENTÃO
        DEVOLVER prazo_normal * 2
    SENÃO
        DEVOLVER prazo_normal

prazo = prazo_final(15, VERDADEIRO)                       ← CHAMADA: manda a função rodar
```

A marca da chamada é o parêntese depois do nome: `prazo_final(15, VERDADEIRO)`. É ele que manda a função rodar com o que está dentro dele.

Definir não executa nada. O que executa é a chamada.

Entrada e saída

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):    ← ENTRADAS: o que a função recebe
    SE tem_prazo_em_dobro ENTÃO
        DEVOLVER prazo_normal * 2                        ← SAÍDA: o que volta para quem chamou
    SENÃO
        DEVOLVER prazo_normal                            ← SAÍDA, pelo outro caminho
```

A definição fixa duas coisas. As entradas, entre parênteses depois do nome, e a saída, na linha do `DEVOLVER`. O que fica entre as duas é a conta.

Uma função se descreve pelo que entra e pelo que sai.

Variável ou valor?

```
resultado = "PROCEDENTE"
```

Nesta linha, qual é a variável e qual é o valor? E qual é a diferença entre os dois?

Variável, valor e tipo

```
resultado = "PROCEDENTE"
```

Elemento	Neste caso
variável	`resultado`, o nome estável do lugar onde registramos
valor	`"PROCEDENTE"`, o conteúdo registrado neste caso
tipo	a regra sobre quais valores cabem ali

Parâmetro e argumento

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):
    ...

prazo = prazo_final(15, VERDADEIRO)
```

Elemento	Neste caso
parâmetro	`prazo_normal`, o nome escrito na definição, como a variável
argumento	`15`, o que entra na chamada, como o valor

A cada chamada, o parâmetro recebe o argumento. Nesta, `prazo_normal` vale 15. Em outra chamada, vale outra coisa.

Abrir o pote, agora sim

Na aula 5, “abre o pote” foi uma falha: parecia uma instrução e era um roteiro inteiro escondido num verbo.

```
FUNÇÃO abrir_o_pote(pote):
    segurar o pote com uma mão
    girar a tampa no sentido anti-horário até soltar
    tirar a tampa e apoiar na mesa
    DEVOLVER o pote aberto

pote_aberto = abrir_o_pote(pote_de_nutella)
```

Depois que a função existe e está escrita, “abrir o pote” deixa de ser falha e vira instrução legítima. O roteiro não sumiu: ganhou nome.

O que sai daqui

O que fica guardado em `p` depois desta chamada?

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):
    SE tem_prazo_em_dobro ENTÃO
        DEVOLVER prazo_normal * 2
    SENÃO
        DEVOLVER prazo_normal

p = prazo_final(5, FALSO)
```

O que sai daqui

RESPOSTA

5.

Na chamada, `prazo_normal` vale 5 e `tem_prazo_em_dobro` vale FALSO. O `SE` cai no `SENÃO`, e a função devolve `prazo_normal` como veio. O `DEVOLVER` é o que sai; sem ele, `p` ficaria com nada.

Esqueci o DEVOLVER

Tirei as duas linhas de `DEVOLVER` e guardei o resultado numa variável. O que fica em `p`?

```
FUNÇÃO prazo_final(prazo_normal, tem_prazo_em_dobro):
    SE tem_prazo_em_dobro ENTÃO
        prazo = prazo_normal * 2
    SENÃO
        prazo = prazo_normal

p = prazo_final(15, VERDADEIRO)
```

Esqueci o DEVOLVER

RESPOSTA

Nada. `prazo` nasceu dentro da função e fica lá.

A função calculou 30 e guardou em `prazo`, mas não entregou nada a quem chamou. `p` recebe o que a função devolve, e ela não devolveu. Quem quer usar o valor depois precisa do `DEVOLVER`.

Dentro e fora da função

DENTRO

Os parâmetros, como `prazo_normal`, e as variáveis criadas ali, como `prazo`. Existem enquanto a função roda e somem quando ela acaba.

FORA

Só o que a função devolve, e só se quem chamou guardar numa variável: `p = prazo_final(15, VERDADEIRO)`.

Só o que é devolvido existe fora da função.

Três fases, três funções

Insper

A dosimetria numa frase

Código Penal. Art. 68, caput.

> **Art. 68 -** A pena-base será fixada atendendo-se ao critério do art. 59 deste Código; em seguida serão consideradas as circunstâncias atenuantes e agravantes; por último, as causas de diminuição e de aumento.

Três fases, separadas por ponto e vírgula. Cada uma recebe o que a anterior produziu.

A saída de uma é a entrada da seguinte

```
FUNÇÃO fixar_pena_base(pena_minima, pena_maxima, circunstancias):
    ... critério do art. 59 ...
    DEVOLVER pena_base

FUNÇÃO aplicar_atenuantes_e_agravantes(pena_base, atenuantes, agravantes):
    ...
    DEVOLVER pena_intermediaria

FUNÇÃO aplicar_causas(pena_intermediaria, diminuicoes, aumentos):
    ...
    DEVOLVER pena_definitiva
```

Três funções, uma por fase. O que a primeira devolve, `pena_base`, é o primeiro parâmetro da segunda; o que a segunda devolve é o primeiro da terceira. As reticências são a regra de cada fase, que ainda vamos escrever.

Criando nossos próprios tipos

Insper

O que um texto sabe fazer

```
resultado = "PROCEDENTE"

resultado.lower()          → "procedente"
"Parcialmente".upper()    → "PARCIALMENTE"
```

Um texto já vem com coisas que sabe fazer: passar para minúsculas, para maiúsculas. Pede-se com ponto e parêntese, como uma chamada de função, só que grudada na coisa.

O que uma tabela carrega e sabe fazer

```
tabela.shape     → (200, 5)
tabela.head()    → as cinco primeiras linhas
```

No módulo 3, a tabela de dados é um tipo, o `DataFrame`. Ela carrega o próprio tamanho, 200 linhas e 5 colunas, e sabe mostrar o começo de si mesma.

`shape` se lê, sem parêntese. `head()` se chama, com parêntese.

Atributo e método

ATRIBUTO

```
tabela.shape
```

Um dado que a coisa carrega. Lê-se, sem parêntese.

MÉTODO

```
resultado.lower()
```

Algo que a coisa faz. Chama-se, com parêntese, como uma função.

Os dois se pedem com ponto. O parêntese diz qual é qual.

Podemos criar os nossos

A CLASSE É O MOLDE

```
CLASSE Cachorro:
    nome      → "Rex"
    raca      → "vira-lata"
    idade     → 3
    latir()   → "au au"
    sentar()  → o cachorro senta
```

CADA CACHORRO É UM OBJETO

```
rex = Cachorro("Rex", "vira-lata", 3)
toto = Cachorro("Totó", "poodle", 8)

rex.nome      → "Rex"
toto.nome     → "Totó"
rex.idade     → 3
toto.idade    → 8
toto.latir()  → "au au"
```

Três atributos e dois métodos, definidos uma vez no molde. Cada cachorro sai dele com os próprios valores: Rex e Totó são dois, do mesmo tipo.

Um tipo jurídico: Processo

```
CLASSE Processo:
    numero                  → "0000000-00.2026.8.26.0100"
    partes                  → [{"nome": "Ana", "papel": "AUTORA"}, ...]
    pecas                   → [{"tipo": "PETIÇÃO INICIAL", ...}, ...]
    data_sentenca           → 01/09/2026
    data_transito           → NULO
    transitou_em_julgado()  → FALSO
    juntar_peca(peca)       → acrescenta peca em pecas
    precisa_revisao()       → VERDADEIRO
```

Cinco atributos, os campos do dicionário da aula 8, e três métodos, as perguntas e ações que se fazem a um processo.

Para que criar um tipo

1. Todo processo saído do molde tem os mesmos campos. No dicionário, `data_transito` podia faltar numa peça e existir na outra.

2. A regra de decidir vive junto do dado, escrita uma vez. Quem tem um processo não refaz a conta: pergunta.

3. O código lê como a frase jurídica:

```
PARA CADA processo EM processos:
    SE processo.precisa_revisao() ENTÃO
        revisar processo
```

Função ou método

FUNÇÃO

```
contar_sem_transito(decisoes)
```

Existe solta. Recebe a lista como entrada.

MÉTODO

```
decisoes.contar_sem_transito()
```

Pertence à coisa. A lista já sabe contar a si mesma.

Um método é uma função que mora dentro de um tipo e age sobre o objeto. O ponto é a marca: `coisa.metodo()`.

Programação orientada a objetos

É o nome dessa forma de programar: o dado e o que se faz com ele andam juntos, num objeto, e o objeto sai de um tipo que nós definimos. Python é orientada a objetos: tudo nela, texto, lista, tabela, é objeto.

1. Com variáveis soltas, `numero`, `partes`, `pecas` de duzentos processos são seiscentas variáveis para não misturar. No objeto, cada processo carrega as suas.

2. Com funções soltas, quem chama precisa passar tudo: `precisa_revisao(data_sentenca, data_transito)`. O método já sabe onde está: `processo.precisa_revisao()`.

3. Mudou a regra? Muda-se num lugar, dentro do tipo, e todo processo passa a decidir do jeito novo.

Amanhã

Na aula de amanhã, quarta 09/09, vamos preparar nosso ambiente para começar a trabalhar com Python.